

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Observation Task No.: 3

Student Name: J Satya Srikar

Roll No.: A24126552259

Date: 18/7/26

1. TITLE

Interactive To-Do List using DOM and Event Listeners

2. AIM

Interactive To-Do List using DOM and Event Listeners

3. OBJECTIVE

The objectives of this observation task are:

- To study interactive lists creations and management via Event listeners.
 - To select input fields, list scopes, and elements markers dynamically from DOM.
 - To validate text entries to prevent empty task listings.
 - To assign class structures dynamically toggling completed lists (line-through styles).
 - To delete elements dynamically from the document memory tree via `remove()` API.
-

4. THEORY

Interactive task managers require DOM selectors, structures creation APIs, and action event listeners. Click handlers are bound to additions buttons, drawing values from inputs, verifying string contents, and creating list nodes. Actions components (Complete, Delete buttons) are bound to lists items, toggling completion styles via `classList` or executing element deletion processes dynamically from visual interfaces, adjusting placeholders.

Theory of DOM Events & Task list nodes:

- **DOM Bindings:** Handled using `document.getElementById()` selectors, mapping input nodes and unordered lists (`<ul>`).

- **Button Event Observers:** Bind click actions using `addEventListener('click', ...)`, prompting validation alerts for blank text entries.
 - **Dynamic Layout generation:** List items (`<li>`) are initialized, adding classes (`task-item`) and appending internal buttons.
 - **State modification:** Toggled using the `classList.toggle('completed')` shorthand, modifying CSS styles.
 - **Dynamic Garbage Cleanup:** Removing completed or unwanted list item nodes from the DOM tree using `remove()`.
-

5. ALGORITHM / PROCEDURE

1. Establish directories: *Observation/ObTask3/src*, *Docs*, and *Screenshots*.
 2. Write *index.html* defining input sections, task lists placeholders, and empty alert markers.
 3. Create *style.css* defining input boundaries, complete/delete button colors, list layouts, and struck completion styles.
 4. Create *script.js*, map elements, and bind a click observer to the Add button.
 5. In event handler, check input content; trigger alert popups on empty entries.
 6. Create list items, insert complete / delete button tags, and bind local event observers to each task card.
 7. Configure complete triggers to toggle class lists, and set delete triggers to run removal scripts.
 8. Test task addition, completion toggling, and list deletion in browser.
-

6. PROGRAM

Files: *index.html* and *style.css* (Side-by-Side Columns)

index.html (Complete) & style.css (Complete) - Double Column

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>My To-Do List</title>
  <link rel="stylesheet" href="./style.css">
</head>
<body>

  <div class="container">

    <h1>My To-Do List</h1>

    <div class="input-section">
      <input
        type="text"
        id="taskInput"
        placeholder="Enter a task..."
      >

      <button id="addTaskBtn">Add Task</button>
    </div>

    <p id="emptyMessage">No tasks available.</p>

    <ul id="taskList"></ul>

  </div>

  <script src="./script.js"></script>

</body>
</html>
```

```
/* style.css */
body {
  font-family: Arial, sans-serif;
  background-color: #f2f2f2;
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
}

.container {
  width: 500px;
  background-color: white;
  padding: 30px;
  border-radius: 10px;
  box-shadow: 0 5px 15px rgba(0, 0, 0, 0.2);
}

h1 {
  text-align: center;
}

.input-section {
  display: flex;
  gap: 10px;
}

#taskInput {
  flex: 1;
  padding: 10px;
  font-size: 16px;
}

button {
  padding: 10px 15px;
  border: none;
  border-radius: 5px;
  cursor: pointer;
}

#addTaskBtn {
  background-color: #2563eb;
  color: white;
}

#taskList {
  list-style: none;
  padding: 0;
}

.task-item {
  display: flex;
  align-items: center;
  gap: 10px;
  padding: 12px;
  margin-top: 10px;
  background-color: #f5f5f5;
  border-radius: 5px;
}

.task-text {
  flex: 1;
}

.completed {
  text-decoration: line-through;
  color: gray;
}

.complete-btn {
  background-color: #22c55e;
  color: white;
}

.delete-btn {
  background-color: #ef4444;
  color: white;
}

#emptyMessage {
  text-align: center;
  color: gray;
  margin-top: 20px;
}
```

script.js (Complete)

```
// Selecting elements from the DOM
const taskInput = document.getElementById("taskInput");
const addTaskBtn = document.getElementById("addTaskBtn");
const taskList = document.getElementById("taskList");
const emptyMessage = document.getElementById("emptyMessage");

// Add Task button event listener
addTaskBtn.addEventListener("click", function () {

    // Get the task entered by the user
    const taskText = taskInput.value.trim();

    // Check if input is empty
    if (taskText === "") {
        alert("Please enter a task!");
        return;
    }

    // Create a new list item
    const listItem = document.createElement("li");

    // Add class to list item
    listItem.classList.add("task-item");

    // Create task text
    const task = document.createElement("span");
    task.classList.add("task-text");
    task.textContent = taskText;

    // Create Complete button
    const completeBtn = document.createElement("button");
    completeBtn.textContent = "Complete";
    completeBtn.classList.add("complete-btn");

    // Create Delete button
    const deleteBtn = document.createElement("button");
    deleteBtn.textContent = "Delete";
    deleteBtn.classList.add("delete-btn");

    // Complete button event listener
    completeBtn.addEventListener("click", function () {

        task.classList.toggle("completed");

    });

    // Delete button event listener
    deleteBtn.addEventListener("click", function () {

        listItem.remove();

        // Show empty message if no tasks remain
        checkEmptyList();

    });

    // Add elements to the list item
    listItem.appendChild(task);
    listItem.appendChild(completeBtn);
    listItem.appendChild(deleteBtn);

    // Add list item to the task list
    taskList.appendChild(listItem);

    // Clear input box
    taskInput.value = "";

    // Hide empty message
    checkEmptyList();

});

// Function to check whether task list is empty
function checkEmptyList() {

    if (taskList.children.length === 0) {
        emptyMessage.style.display = "block";
    } else {
        emptyMessage.style.display = "none";
    }

}
```

7. CODE EXPLANATION

DOM Statement / Selector	Explanation
<code>taskInput.value.trim()</code>	Reads text inputs value and trims out starting / trailing whitespaces.
<code>document.createElement("li")</code>	Creates a new list item node dynamically inside browser memory.
<code>listItem.classList.add("task-item")</code>	Attaches CSS styling selectors to lists item containers.
<code>task.classList.toggle("completed")</code>	Toggles strike-through style configurations on text spans upon clicking complete.
<code>listItem.remove()</code>	Deletes lists item nodes dynamically from document structures.
<code>taskList.appendChild(listItem)</code>	Inserts lists items into active task lists wrappers.
<code>taskList.children.length === 0</code>	Inspects number of active lists nodes inside task lists wrapper.
<code>emptyMessage.style.display = "block"</code>	Toggles block visibility on lists empty messages segments.

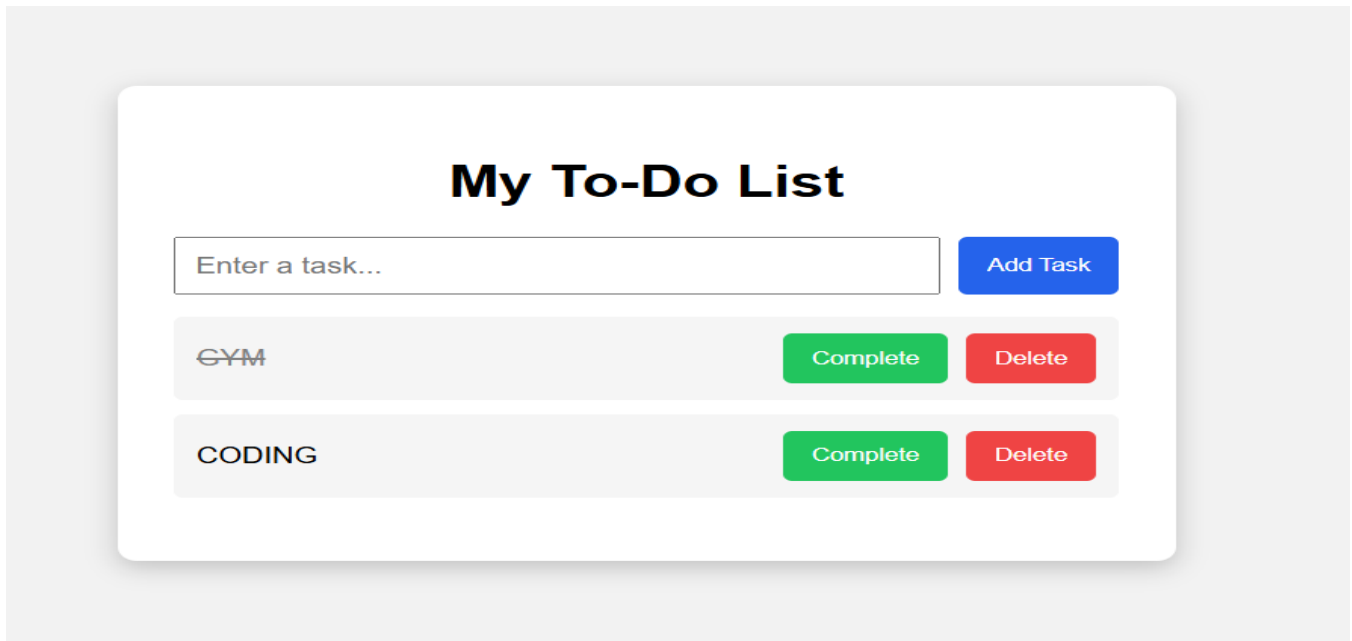
8. TEST CASES AND EXECUTION DATA

The following manual execution tests verify the To-Do task actions in Google Chrome:

TC ID	Test Description	Input / Action	Expected Result	Status
TC-01	Empty Inputs Block	Click Add Task with empty text box	Alert popup triggers, prompting to enter text	Pass
TC-02	Tasks Additions	Type 'Revise JS' and click Add Task	Task appends to list; empty list placeholder hides	Pass
TC-03	Tasks Completions	Click Complete on 'Revise JS'	Task text strikes through and changes font color to gray	Pass
TC-04	Tasks Deletion	Click Delete on 'Revise JS'	Task element is removed from DOM grid	Pass
TC-05	Empty Placeholder Restore	Remove all lists items	Empty alert text 'No tasks available.' displays correctly	Pass

9. OUTPUT

Output: Rendered Interactive To-Do List Application



10. RESULT

Thus, the interactive To-Do application (incorporating DOM selectors, click event handlers, dynamically created lists nodes, completion toggle stylings, and element removal commands) was successfully designed, implemented, and verified.

Submission Package Structure:

Observation/ObTask3/

■ ■ ■ ■ **Docs/**

■ ■ ■ ■ ObsWeek3.pdf

■ ■ ■ ■ **Screenshots/**

■ ■ ■ ■ image.png

■ ■ ■ ■ **src/**

■ ■ ■ ■ index.html

■ ■ ■ ■ script.js

■ ■ ■ ■ style.css